

HOSPITAL MANAGEMENT SYSTEM (HMS)

Phase 1 Implementation: Code Review & Viva Preparation Guide

TOPIC 1: SECURITY FOUNDATION

A. Environment Variables

1. What was implemented:

Critical system secrets (SECRET_KEY, JWT_SIGNING_KEY, ENCRYPTION_KEY, DATABASE_URL) were detached from codebase source files and offloaded to an uncommitted .env file loaded dynamically using python-decouple.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 10-27)
- clinic_backend/.env

3. Exact Code Location & Snippet:

File: clinic_backend/settings.py | Lines: 10–27

```
SECRET_KEY = config("SECRET_KEY")
JWT_SIGNING_KEY = config("JWT_SIGNING_KEY", default=SECRET_KEY)
if JWT_SIGNING_KEY == SECRET_KEY:
    raise ImproperlyConfigured("SECRET_KEY and JWT_SIGNING_KEY must be different for enhanced security.")

ENCRYPTION_KEY = config("ENCRYPTION_KEY", default=None)
if not ENCRYPTION_KEY:
    raise ImproperlyConfigured("ENCRYPTION_KEY environment variable is required and missing.")
```

4. Flow Explanation:

Server Startup → settings.py imported → decouple.config() reads .env → Key validation & Fernet check → App initializes with verified keys.

5. Why this code was added (Real-World HMS Justification):

In a Healthcare application handling sensitive electronic health records (EHR), hardcoding secret keys or database credentials in version control exposes patient records to key leakage, regulatory fines (HIPAA/DISHA), and unauthorized token forgery.

6. Before vs After:

Before: Secret keys hardcoded in settings.py committed to Git repository.

After: Missing environment variables or using identical keys for Django and JWT immediately aborts server boot with ImproperlyConfigured.

7. Examiner Viva Questions & Answers:

Q: Why use JWT_SIGNING_KEY separate from Django's SECRET_KEY?

A: Key isolation. If SECRET_KEY is compromised in web session signing, an attacker cannot forge JWT authentication tokens for API requests.

Q: How does python-decouple differ from os.environ.get()?

A: python-decouple automatically reads from local .env files during development, supports type casting (e.g. cast=bool), and raises runtime errors for missing required variables.

Q: What happens if ENCRYPTION_KEY is missing at server startup?

A: The system raises ImproperlyConfigured and refuses to launch, ensuring no patient data is written without encryption.

B. PostgreSQL Configuration

1. What was implemented:

Enforced PostgreSQL as the primary relational database via DATABASE_URL parsing with connection pooling and health checks enabled.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 118-138)

3. Exact Code Location & Snippet:

File: clinic_backend/settings.py | Lines: 118–138

```
DATABASE_URL = config("DATABASE_URL", default=None)
if not DATABASE_URL:
    raise ImproperlyConfigured("DATABASE_URL environment variable is required and missing.")

db_config = dj_database_url.parse(DATABASE_URL, conn_max_age=600, conn_health_checks=True)
if "postgres" not in db_config.get("ENGINE", ""):
    raise ImproperlyConfigured("Only PostgreSQL database (`postgresql://` or `postgres://`) is supported.")
```

4. Flow Explanation:

Request → Django ORM → Connection Pool (conn_max_age=600) → PostgreSQL Database Connection → Health check ping → Query Execution.

5. Why this code was added (Real-World HMS Justification):

Hospitals process concurrent operations (lab reports, appointment bookings, billing). PostgreSQL provides row-level locking (SELECT FOR UPDATE), transaction isolation, and ACID guarantees necessary to prevent race conditions during slot bookings.

6. Before vs After:

Before: Default SQLite file DB used, vulnerable to 'database is locked' errors during concurrent access.

After: Production backend mandates PostgreSQL, reusing persistent connections for up to 10 minutes (600s).

7. Examiner Viva Questions & Answers:

Q: What does conn_max_age=600 achieve?

A: Enables connection pooling by keeping DB connections open for up to 600 seconds, avoiding the TCP overhead of opening/closing a connection per HTTP request.

Q: What is conn_health_checks=True?

A: Django pings the DB before reusing a pooled connection to ensure the connection is alive, preventing 500 errors on broken sockets.

C. Production Settings

1. What was implemented:

Configured production guardrails including dynamic DEBUG toggle, strict ALLOWED_HOSTS, WhiteNoise static file compression, and SSL redirection.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 26-29, 164-167)

3. Exact Code Location & Snippet:

File: clinic_backend/settings.py | Lines: 26–29 & 164–167

```

DEBUG = config("DEBUG", default=False, cast=bool)
ALLOWED_HOSTS = [host.strip() for host in config("ALLOWED_HOSTS", default="").split(",") if host.strip()]

if not DEBUG:
    STATICFILES_STORAGE = "whitenoise.storage.CompressedManifestStaticFilesStorage"
    MIDDLEWARE.insert(1, "whitenoise.middleware.WhiteNoiseMiddleware")

```

4. Flow Explanation:

HTTP Request → WhiteNoiseMiddleware (intercepts static files in prod) → ALLOWED_HOSTS check → App processing → Error response (Sanitized tracebacks when DEBUG=False).

5. Why this code was added (Real-World HMS Justification):

Running DEBUG=True in production leaks internal stack traces, DB queries, and secret keys to end users upon application exceptions.

6. Before vs After:

Before: Hardcoded DEBUG = True and open host headers (ALLOWED_HOSTS = ["*"]).

After: Environment-driven DEBUG state with manifest-hashed, compressed static asset delivery in production.

7. Examiner Viva Questions & Answers:

Q: Why is WhiteNoise used instead of django.contrib.staticfiles in production?

A: Django's static file server is unsuitable for production. WhiteNoise serves static files directly from Gunicorn/WSGI with cache-forever headers and compression.

Q: What security risk does DEBUG=True introduce?

A: It reveals full tracebacks, local variables, settings values, and SQL query contents on any 500 server error page.

D. CORS (Cross-Origin Resource Sharing)

1. What was implemented:

Restricted API cross-origin HTTP access strictly to validated frontend origins while enabling secure credential transmission.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 88, 212-218)

3. Exact Code Location & Snippet:

File: clinic_backend/settings.py | Lines: 212–218

```

CORS_ALLOW_ALL_ORIGINS = False
CORS_ALLOW_CREDENTIALS = True
CORS_ALLOWED_ORIGINS = [
    origin.strip()
    for origin in config("CORS_ALLOWED_ORIGINS", default="").split(",")
    if origin.strip()
]

```

4. Flow Explanation:

Browser Preflight OPTIONS Request → CorsMiddleware → Checks Origin header against CORS_ALLOWED_ORIGINS → Appends Access-Control-Allow-Credentials: true → Preflight Approved → Actual API Request.

5. Why this code was added (Real-World HMS Justification):

Without CORS restrictions, malicious third-party websites could execute API calls against the hospital backend using a logged-in user's browser context. CORS_ALLOW_CREDENTIALS = True is required to allow sending cookies cross-origin.

6. Before vs After:

Before: Open CORS policy allowing arbitrary web origins to send requests.

After: Explicit whitelist enforcement populated dynamically via environment configuration.

7. Examiner Viva Questions & Answers:

Q: Why must `CORS_ALLOW_ALL_ORIGINS` be False when `CORS_ALLOW_CREDENTIALS` is True?

A: Browsers reject credentialed requests (with `Credentials: true` / cookies) if `Access-Control-Allow-Origin` is set to `*`. Origin whitelisting is mandatory.

E. CSRF (Cross-Site Request Forgery) Protection

1. What was implemented:

Configured trusted origins for CSRF checks and implemented an explicit endpoint (`CSRFGeneratorView`) that sets the CSRF token cookie on client application startup.

2. Files Modified:

- `clinic_backend/clinic_backend/settings.py` (Lines 90, 221-225, 230-235)
- `clinic_backend/accounts/views.py` (Lines 518-523)

3. Exact Code Location & Snippet:

Class: `CSRFGeneratorView` in `accounts/views.py` | Lines: 518–523

```
@method_decorator(ensure_csrf_cookie, name='dispatch')
class CSRFGeneratorView(APIView):
    permission_classes = [AllowAny]

    def get(self, request):
        return Response({"csrfToken": get_token(request)})
```

4. Flow Explanation:

Frontend Initialization → GET `/api/accounts/csrf/` → `@ensure_csrf_cookie` executes → `csrftoken` Cookie set in browser → Frontend attaches `X-CSRFToken` header → `CsrfViewMiddleware` validates tokens.

5. Why this code was added (Real-World HMS Justification):

Since authentication credentials (refresh tokens) are stored in `HttpOnly` cookies, browser calls are vulnerable to CSRF attacks where an external site submits unauthorized state-changing forms.

6. Before vs After:

Before: Unprotected cookie-authenticated REST endpoints.

After: Mandatory CSRF cookie distribution and trusted origin checking for state-changing HTTP requests.

7. Examiner Viva Questions & Answers:

Q: Why is `CSRF_COOKIE_HTTPONLY` set to False?

A: The frontend JavaScript framework must be able to read the CSRF cookie value to send it back in the `X-CSRFToken` request header.

Q: What decorator ensures the CSRF cookie is set on response?

A: `@ensure_csrf_cookie`.

F. Security Headers

1. What was implemented:

Configured HTTP security headers including HSTS, Frame Options, XSS protection, and HTTPS redirection.

2. Files Modified:

- `clinic_backend/clinic_backend/settings.py` (Lines 227-245, 271)

3. Exact Code Location & Snippet:

File: `clinic_backend/settings.py` | Lines: 227–245 & 271

```

SECURE_PROXY_SSL_HEADER = ("HTTP_X_FORWARDED_PROTO", "https")
SECURE_SSL_REDIRECT = config("SECURE_SSL_REDIRECT", default=not DEBUG, cast=bool)
SESSION_COOKIE_SECURE = config("SESSION_COOKIE_SECURE", default=not DEBUG, cast=bool)
CSRF_COOKIE_SECURE = config("CSRF_COOKIE_SECURE", default=not DEBUG, cast=bool)
SESSION_COOKIE_HTTPONLY = True
CSRF_COOKIE_HTTPONLY = False

if not DEBUG:
    SECURE_HSTS_SECONDS = config("SECURE_HSTS_SECONDS", default=31536000, cast=int)
    SECURE_HSTS_INCLUDE_SUBDOMAINS = True
    SECURE_HSTS_PRELOAD = True
SECURE_CROSS_ORIGIN_OPENER_POLICY = "same-origin-allow-popups"

```

4. Flow Explanation:

Client Request → SecurityMiddleware / XFrameOptionsMiddleware → Enforces HTTPS redirect → Injects Strict-Transport-Security, X-Frame-Options: DENY, X-Content-Type-Options: nosniff headers into response.

5. Why this code was added (Real-World HMS Justification):

Prevents clickjacking framing attacks, Man-In-The-Middle (MITM) downgrade attacks, and passive cookie sniffing over unencrypted connections.

6. Before vs After:

Before: Default headers lacking strict transport security or cookie secure flags.

After: Automated production HTTPS enforcement, HSTS preloading for 1 year, and cookie transmission limited strictly to encrypted channels.

7. Examiner Viva Questions & Answers:

Q: What does HSTS (SECURE_HSTS_SECONDS) do?

A: It instructs client browsers to strictly convert all future HTTP requests to HTTPS automatically before sending them over the network.

G. Password Policy

1. What was implemented:

Configured Django password validation policies (minimum 10 characters, similarity checks, common password checks, numeric checks) and custom password strength checks on user creation endpoints.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 141-153)
- clinic_backend/accounts/views.py (Lines 258-270)

3. Exact Code Location & Snippet:

File: settings.py (L141-153) & UserViewSet.create_doctor in accounts/views.py (L258-270)

```

# settings.py
AUTH_PASSWORD_VALIDATORS = [
    {"NAME": "django.contrib.auth.password_validation.UserAttributeSimilarityValidator"},
    {"NAME": "django.contrib.auth.password_validation.MinimumLengthValidator", "OPTIONS": {"min_length": 10}},
    {"NAME": "django.contrib.auth.password_validation.CommonPasswordValidator"},
    {"NAME": "django.contrib.auth.password_validation.NumericPasswordValidator"},
]

# accounts/views.py
try:
    password_validation.validate_password(password)
except ValidationError as e:
    return Response({"non_field_errors": list(e.messages)}, status=status.HTTP_400_BAD_REQUEST)

```

4. Flow Explanation:

Password Submission → Passed to `password_validation.validate_password()` → Checks min length (10 chars), dictionary commonality, user attribute similarity → Raises `ValidationError` if weak → Saves password hashed via PBKDF2.

5. Why this code was added (Real-World HMS Justification):

Healthcare staff accounts have access to sensitive records. Weak passwords make the hospital network susceptible to brute-force and credential-stuffing attacks.

6. Before vs After:

Before: Default 8-character minimum without custom backend validation in management APIs.

After: Enforced minimum 10-character length across all registration and admin creation endpoints.

7. Examiner Viva Questions & Answers:

Q: How does Django store user passwords in the database?

A: Using PBKDF2 with SHA-256 password hashing and a unique salt per user (never plaintext).

H. Rate Limiting

1. What was implemented:

Configured global REST Framework throttling to restrict Anonymous requests to 60/min and Authenticated requests to 1000/min.

2. Files Modified:

- `clinic_backend/clinic_backend/settings.py` (Lines 187-194)

3. Exact Code Location & Snippet:

File: `clinic_backend/settings.py` | Lines: 187–194

```
"DEFAULT_THROTTLE_CLASSES": [  
    "rest_framework.throttling.AnonRateThrottle",  
    "rest_framework.throttling.UserRateThrottle",  
],  
"DEFAULT_THROTTLE_RATES": {  
    "anon": config("THROTTLE_ANON_RATE", default="60/minute"),  
    "user": config("THROTTLE_USER_RATE", default="1000/minute"),  
},
```

4. Flow Explanation:

API Request → Throttle class checks client IP / User ID → Compares request count against time bucket (60s) → Exceeds threshold? → Returns HTTP 429 Too Many Requests.

5. Why this code was added (Real-World HMS Justification):

Protects public authentication endpoints (login, registration) from brute-force attempts and denial-of-service (DoS) attacks.

6. Before vs After:

Before: Unthrottled API endpoints.

After: Automated rate limiting per IP for unauthenticated users and per user ID for logged-in sessions.

7. Examiner Viva Questions & Answers:

Q: What HTTP status code is returned when a rate limit is exceeded?

A: 429 Too Many Requests.

TOPIC 2: AUTHENTICATION

A. JWT Authentication & Access Tokens

1. What was implemented:

Stateless authentication mechanism issuing short-lived signed JSON Web Tokens (15-minute access token) upon successful credential verification.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 181-183, 199-209)
- clinic_backend/accounts/views.py (Lines 116-146)

3. Exact Code Location & Snippet:

Method: UserViewSet.login in accounts/views.py | Lines: 135–146

```
refresh = RefreshToken.for_user(user)
response = Response(
    {
        "message": "Login successful",
        "user": UserSerializer(user).data,
        "tokens": {
            "access": str(refresh.access_token),
        },
    },
    status=status.HTTP_200_OK,
)
return set_auth_cookie(response, refresh)
```

4. Flow Explanation:

Login Credentials POSTed → Authenticated → RefreshToken.for_user() → Generates short-lived Access Token & Refresh Token → Access token returned in JSON payload → Refresh token stored in HttpOnly cookie.

5. Why this code was added (Real-World HMS Justification):

JWTs allow stateless horizontal scaling across hospital backend services without DB session lookups on every single API request.

6. Before vs After:

Before: Session-based auth requiring server session storage lookups.

After: Statelessly validated access tokens signed cryptographically via JWT_SIGNING_KEY.

7. Examiner Viva Questions & Answers:

Q: What is inside a JWT payload?

A: User ID (user_id), expiration time (exp), issued time (iat), and token type (access/refresh), cryptographically signed with JWT_SIGNING_KEY.

B. HttpOnly Cookies & Refresh Tokens

1. What was implemented:

Implemented secure cookie utility functions to write the long-lived refresh_token into an HttpOnly, SameSite, Secure cookie, isolating refresh tokens from browser JavaScript context.

2. Files Modified:

- clinic_backend/accounts/cookie_utils.py (Lines 1-28)
- clinic_backend/accounts/views.py (Lines 112, 146, 234)

3. Exact Code Location & Snippet:

File: accounts/cookie_utils.py | Lines: 5–18

```
def set_auth_cookie(response, refresh_token):
    max_age = settings.SIMPLE_JWT.get('REFRESH_TOKEN_LIFETIME', timedelta(days=7)).total_seconds()
    is_secure = getattr(settings, 'SESSION_COOKIE_SECURE', False)
    samesite = getattr(settings, 'SESSION_COOKIE_SAMESITE', 'Lax')

    response.set_cookie(
        key='refresh_token',
        value=str(refresh_token),
        max_age=int(max_age),
        secure=is_secure,
        httponly=True,
        samesite=samesite
    )
    return response
```

4. Flow Explanation:

Login/Refresh Endpoint → set_auth_cookie() → Sets Set-Cookie: refresh_token=...; HttpOnly; Secure; SameSite=Lax header → Browser stores cookie securely and attaches it automatically to refresh endpoint calls.

5. Why this code was added (Real-World HMS Justification):

If refresh tokens are stored in localStorage or sessionStorage, any Cross-Site Scripting (XSS) vulnerability allows attackers to extract persistent credentials and gain unauthorized access to patient data.

6. Before vs After:

Before: Tokens sent and stored entirely in frontend memory/local storage.

After: Long-lived refresh token protected behind browser HttpOnly flag.

7. Examiner Viva Questions & Answers:

Q: Why can't JavaScript access an HttpOnly cookie?

A: Browser engines prevent document.cookie or client script APIs from reading cookies marked with the HttpOnly flag.

C. Token Rotation & Token Blacklisting

1. What was implemented:

Configured SimpleJWT to issue a new refresh token every time the access token is refreshed, while instantly blacklisting the spent refresh token in the database.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 199-209)
- clinic_backend/accounts/views.py (Lines 486-516)

3. Exact Code Location & Snippet:

File: settings.py (L202-203) & LogoutView in accounts/views.py (L512-514)

```
# settings.py
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(minutes=15),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "ROTATE_REFRESH_TOKENS": True,
    "BLACKLIST_AFTER_ROTATION": True,
}

# accounts/views.py LogoutView
if refresh_token:
    try:
        token = RefreshToken(refresh_token)
        token.blacklist()
    except Exception: pass
```

4. Flow Explanation:

Access Token Expires (after 15 min) → Client POSTs to `/api/accounts/token/refresh/` with refresh_token HttpOnly cookie → CookieTokenRefreshView validates token → Old refresh token blacklisted in DB table → New access token & rotated refresh token issued.

5. Why this code was added (Real-World HMS Justification):

Prevents replay attacks. If a refresh token is leaked or intercepted, it can only be used once before being invalidated upon rotation.

6. Before vs After:

Before: Static refresh tokens reusable for 7 days until expiry.

After: Single-use refresh tokens invalidated immediately upon exchange or logout.

7. Examiner Viva Questions & Answers:

Q: What Django model tracks blacklisted tokens?

A: `rest_framework_simplejwt.token_blacklist.models.BlacklistedToken`.

D. Logout & Session Restore

1. What was implemented:

Implemented explicit server-side token blacklisting and cookie clearance on Logout, alongside cookie-driven token refresh for transparent session restoration on page reload.

2. Files Modified:

- `clinic_backend/accounts/cookie_utils.py` (Lines 20-27)
- `clinic_backend/accounts/views.py` (Lines 486-517)

3. Exact Code Location & Snippet:

Classes: CookieTokenRefreshView & LogoutView in `accounts/views.py` | Lines: 486–517

```
class CookieTokenRefreshView(TokenRefreshView):
    def post(self, request, *args, **kwargs):
        refresh_token = request.COOKIES.get("refresh_token")
        if not refresh_token:
            return Response({"error": "Refresh token not found"}, status=status.HTTP_401_UNAUTHORIZED)
        request.data["refresh"] = refresh_token
        response = super().post(request, *args, **kwargs)
        if "refresh" in response.data:
            refresh_token = response.data.pop("refresh")
            response = set_auth_cookie(response, refresh_token)
        return response

class LogoutView(APIView):
    def post(self, request):
        refresh_token = request.COOKIES.get("refresh_token")
        response = Response({"message": "Logout successful"}, status=status.HTTP_200_OK)
        if refresh_token:
            try:
                token = RefreshToken(refresh_token)
                token.blacklist()
            except Exception:
                pass
        return delete_auth_cookie(response)
```

4. Flow Explanation:

Session Restore: Page Refresh → SPA calls `/api/accounts/token/refresh/` → Cookie extracted automatically → Validated → New Access Token returned.

Logout: Click Logout → POST `/api/accounts/logout/` → Refresh token blacklisted in DB → Cookie deleted via Set-Cookie max-age=0.

5. Why this code was added (Real-World HMS Justification):

Ensures that logging out completely invalidates active credentials on the backend rather than just discarding client-side variables.

6. Before vs After:

Before: Logging out only cleared local memory; refresh tokens remained valid on backend.

After: Immediate server-side blacklisting + secure cookie cleanup.

7. Examiner Viva Questions & Answers:

Q: How does `delete_auth_cookie` purge the cookie from the user's browser?

A: By returning an HTTP header setting the cookie's `max_age=0` and expiration date to the past.

TOPIC 3: AUTHORIZATION

A. Role-Based Permissions (RBAC)

1. What was implemented:

Custom REST Framework permission classes matching user roles (ADMIN, DOCTOR, STAFF, PATIENT, LAB_TECHNICIAN).

2. Files Modified:

- `clinic_backend/accounts/permissions.py` (Lines 1-96)

3. Exact Code Location & Snippet:

Classes: `IsAdmin`, `IsDoctor`, `IsPatient`, etc. in `accounts/permissions.py` | Lines: 6–56

```
class IsAdmin(BasePermission):
    def has_permission(self, request, view):
        return (
            request.user
            and request.user.is_authenticated
            and request.user.role == "ADMIN"
        )

class IsDoctor(BasePermission):
    def has_permission(self, request, view):
        return (
            request.user
            and request.user.is_authenticated
            and request.user.role == "DOCTOR"
        )
```

4. Flow Explanation:

Request → ViewSet execution → `check_permissions()` → Evaluates `has_permission()` on configured classes → Returns False? → HTTP 403 Forbidden.

5. Why this code was added (Real-World HMS Justification):

In a hospital setting, patients cannot perform administrative operations (e.g. creating doctors, viewing hospital billing summaries), and lab technicians must not alter doctor prescriptions.

6. Before vs After:

Before: Generic `IsAuthenticated` check allowing any logged-in user access to all endpoints.

After: Fine-grained role checks strictly enforced per endpoint operation.

7. Examiner Viva Questions & Answers:

Q: Which method is overridden for view-level permissions in DRF?

A: `has_permission(self, request, view)`.

B. Object-Level Permissions

1. What was implemented:

Granular access control validating whether an authenticated user owns or is directly related to a specific database record object instance.

2. Files Modified:

- clinic_backend/accounts/permissions.py (Lines 102-168)

3. Exact Code Location & Snippet:

Classes: IsPatientOwner, IsAppointmentParticipant, IsDoctorOfPatient in accounts/permissions.py | Lines: 102–168

```
class IsAppointmentParticipant(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.user.role in ("ADMIN", "STAFF"):
            return True
        if request.user.role == "PATIENT":
            return obj.patient.user == request.user
        if request.user.role == "DOCTOR":
            return obj.doctor.user == request.user
        return False
```

4. Flow Explanation:

GET /api/appointments/42/ → View retrieves object instance → Calls check_object_permissions(request, obj) → Checks if user is assigned patient or doctor → Disallowed user receives HTTP 403.

5. Why this code was added (Real-World HMS Justification):

Prevents IDOR (Insecure Direct Object Reference) vulnerabilities where Patient A changes the ID in the URL to view Patient B's appointment or medical history.

6. Before vs After:

Before: Any authenticated patient could view any appointment by guessing its integer primary key.

After: Strict object-level relationship check before record retrieval or modification.

7. Examiner Viva Questions & Answers:

Q: What is the difference between has_permission and has_object_permission?

A: has_permission runs before the view action is executed; has_object_permission runs after the object is retrieved from the database via get_object().

C. Queryset Filtering

1. What was implemented:

Server-side data scoping inside get_queryset() to ensure API list views only query and return records relevant to the requesting user's role.

2. Files Modified:

- clinic_backend/appointments/views.py (Lines 34-63)
- clinic_backend/patients/views.py (Lines 65-93)

3. Exact Code Location & Snippet:

Method: AppointmentViewSet.get_queryset in appointments/views.py | Lines: 34–63

```
def get_queryset(self):
    user = self.request.user
    if user.role == "PATIENT":
        queryset = Appointment.objects.filter(patient__user=user)
    elif user.role == "DOCTOR":
        queryset = Appointment.objects.filter(
            doctor__user=user, status__in=["APPROVED", "VISITED"]
        )
    elif user.role in ["ADMIN", "STAFF"]:
        queryset = Appointment.objects.all()
    return queryset.select_related("patient__user", "doctor__user")
```

4. Flow Explanation:

GET /api/appointments/ → get_queryset() called → User role checked → Patient gets WHERE patient_id = X; Doctor gets WHERE doctor_id = Y AND status IN ('APPROVED', 'VISITED') → Query executed.

5. Why this code was added (Real-World HMS Justification):

Prevents data exposure at the collection level. Even if a patient requests the list endpoint /api/appointments/, SQL-level filtering guarantees they only receive their own appointments.

6. Before vs After:

Before: Appointment.objects.all() returned all hospital records to any authenticated user.

After: Automatic DB-level query scoping based on request.user.role.

7. Examiner Viva Questions & Answers:

Q: Why add select_related() to get_queryset()?

A: It performs an SQL JOIN to fetch foreign key relations (patient, doctor user profiles) in a single query, preventing N+1 database query performance bottlenecks.

TOPIC 4: APPOINTMENT INTEGRITY

A. Database Constraints

1. What was implemented:

Added a conditional database constraint UniqueConstraint on the Appointment table to enforce slot uniqueness for active appointments at the PostgreSQL level.

2. Files Modified:

- clinic_backend/appointments/models.py (Lines 35-41)

3. Exact Code Location & Snippet:

File: Appointment.Meta in appointments/models.py | Lines: 35–41

```
class Meta:
    db_table = "appointments"
    ordering = ["-appointment_date", "-appointment_time"]
    constraints = [
        models.UniqueConstraint(
            fields=["doctor", "appointment_date", "appointment_time"],
            condition=models.Q(status__in=["PENDING", "APPROVED"]),
            name="unique_active_appointment_slot"
        )
    ]
```

4. Flow Explanation:

Attempted Duplicate Booking → INSERT INTO appointments (doctor_id, date, time, status='PENDING') → PostgreSQL checks unique_active_appointment_slot index → Slot exists for status PENDING/APPROVED? → DB aborts transaction with SQL IntegrityError.

5. Why this code was added (Real-World HMS Justification):

Application-level checks (if exists()) are subject to race conditions under high concurrency. Database constraints enforce data integrity at the storage layer.

6. Before vs After:

Before: Cancelled or rejected appointments permanently blocked a time slot because of plain unique constraints.

After: Conditional index allows cancelled/rejected time slots to be re-booked while enforcing uniqueness for active bookings (PENDING, APPROVED).

7. Examiner Viva Questions & Answers:

Q: What is a conditional UniqueConstraint in Django?

A: A database constraint that applies a UNIQUE check only to rows matching a specific Q() condition (e.g. status IN ('PENDING', 'APPROVED')).

B. Database Transactions & Locking

1. What was implemented:

Wrapped appointment creation, updates, approvals, and cancellations inside explicit transaction.atomic() blocks using select_for_update() row locking.

2. Files Modified:

- clinic_backend/appointments/views.py (Lines 82-108, 165-190)

3. Exact Code Location & Snippet:

Method: AppointmentViewSet.approve in appointments/views.py | Lines: 165–176

```
with transaction.atomic():
    appointment = Appointment.objects.select_for_update().get(pk=pk)
    if appointment.status != "PENDING":
        return Response({"error": f"Cannot approve appointment in {appointment.status} state."}, status=status.HTTP_409_CONFLICT)
    appointment.status = "APPROVED"
    appointment.save()
    Notification.objects.create(...)
```

4. Flow Explanation:

API Action → Opens DB transaction (transaction.atomic()) → Locks appointment row (SELECT ... FOR UPDATE) → Validates state transition → Updates appointment & creates notifications → Transaction commits.

5. Why this code was added (Real-World HMS Justification):

If two administrators attempt to approve/cancel the same appointment simultaneously, row locking ensures sequential execution and prevents inconsistent notification/status states.

6. Before vs After:

Before: Unprotected DB writes prone to partial failures and race conditions.

After: All-or-nothing atomic transactions with pessimistic row-level database locking.

7. Examiner Viva Questions & Answers:

Q: What does select_for_update() do in PostgreSQL?

A: It locks the selected rows until the end of the current transaction block, preventing other transactions from updating or locking those rows.

C. Duplicate Booking Prevention & IntegrityError Handling

1. What was implemented:

Dual-layer validation: pre-validation in AppointmentSerializer.validate() plus runtime django.db.IntegrityError catching in perform_create() and perform_update().

2. Files Modified:

- clinic_backend/appointments/serializers.py (Lines 122-141)
- clinic_backend/appointments/views.py (Lines 105-108)

3. Exact Code Location & Snippet:

File: AppointmentSerializer.validate in serializers.py (L122-141) & views.py (L105-108)

```
# appointments/serializers.py
def validate(self, data):
    if status in ["PENDING", "APPROVED"] and doctor and date and time:
        qs = Appointment.objects.filter(
            doctor=doctor, appointment_date=date, appointment_time=time, status__in=["PENDING", "APPROVED"]
        )
        if self.instance:
            qs = qs.exclude(id=self.instance.id)
        if qs.exists():
            raise serializers.ValidationError({"non_field_errors": ["This time slot is already booked for the selecte
d doctor."]})
        return data

# appointments/views.py perform_create
except IntegrityError:
    raise serializers.ValidationError({"non_field_errors": ["This time slot is already booked. Please choose another.
"]})
```

4. Flow Explanation:

User submits booking request → Serializer checks existing active slots for doctor → Passes serializer check → DB insertion executes → If concurrent request inserted slot milliseconds earlier, DB constraint triggers IntegrityError → View catches exception and transforms to HTTP 400 validation response.

5. Why this code was added (Real-World HMS Justification):

Prevents double-booking doctor time slots and prevents raw database crash exceptions (500 Internal Server Error) from reaching the frontend user.

6. Before vs After:

Before: Concurrent bookings resulted in unhandled 500 server crashes or double-booked slots.

After: Graceful HTTP 400 response indicating slot unavailability.

7. Examiner Viva Questions & Answers:

Q: Why combine serializer validation with IntegrityError handling?

A: Serializer validation catches 99% of user errors early. Catching IntegrityError acts as the safety net for concurrent race conditions that pass serializer validation simultaneously.

TOPIC 5: AUDIT LOGGING

A. Middleware & Request ID

1. What was implemented:

Created AuditMiddleware to attach a unique UUID (request_id) to every incoming request, echo it in response headers (X-Request-ID), and log structured audit details.

2. Files Modified:

- clinic_backend/audit/middleware.py (Lines 1-60)
- clinic_backend/clinic_backend/settings.py (Line 94)

3. Exact Code Location & Snippet:

Class: AuditMiddleware in audit/middleware.py | Lines: 8–59

```
class AuditMiddleware(MiddlewareMixin):
    def process_request(self, request):
        request.request_id = str(uuid.uuid4())
        return None

    def process_response(self, request, response):
        if hasattr(request, 'request_id'):
            response['X-Request-ID'] = request.request_id
            # ... log AuditEvent ...
        return response
```

4. Flow Explanation:

HTTP Request → AuditMiddleware.process_request() assigns request.request_id = UUID4 → API processing → process_response() appends X-Request-ID header & records AuditEvent → Returns response.

5. Why this code was added (Real-World HMS Justification):

In medical record management, end-to-end tracing is necessary for debugging API calls across distributed frontend/backend logs and complying with legal auditing mandates.

6. Before vs After:

Before: No correlation IDs between client HTTP calls and server log entries.

After: Every response carries a unique X-Request-ID correlated to server logs and audit database records.

7. Examiner Viva Questions & Answers:

Q: What is the purpose of MiddlewareMixin in Django?

A: It provides backward and forward compatibility for Django middleware classes supporting both process_request and process_response methods.

B. Audit Log Model

1. What was implemented:

Created the AuditEvent database model to store permanent logs of user activity, target resources, client IPs, user agents, HTTP methods, and status codes.

2. Files Modified:

- clinic_backend/audit/models.py (Lines 1-28)

3. Exact Code Location & Snippet:

Class: AuditEvent in audit/models.py | Lines: 4–24

```
class AuditEvent(models.Model):
    request_id = models.UUIDField(db_index=True)
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.SET_NULL, null=True, blank=True)
    role = models.CharField(max_length=50, blank=True, null=True)
    action = models.CharField(max_length=255, blank=True, null=True)
    method = models.CharField(max_length=10, blank=True, null=True)
    path = models.CharField(max_length=1024, blank=True, null=True)
    status = models.IntegerField(blank=True, null=True)
    resource = models.CharField(max_length=255, blank=True, null=True)
    ip_address = models.GenericIPAddressField(blank=True, null=True)
    created_at = models.DateTimeField(auto_now_add=True, db_index=True)
```

4. Flow Explanation:

API Request completes → Middleware collects request metadata (User, Role, Method, IP, Resource) → Inserts record into audit_events table → Indexed by request_id and created_at.

5. Why this code was added (Real-World HMS Justification):

Medical compliance regulations demand strict record tracking: who accessed which patient file, when, from what IP address, and whether the access attempt succeeded.

6. Before vs After:

Before: No persistent record of user actions or data access events.

After: Immutable database audit log of every incoming HTTP request.

7. Examiner Viva Questions & Answers:

Q: Why set on_delete=models.SET_NULL on the user foreign key?

A: If a user account is deleted from the system, the historical audit log must remain intact for security compliance rather than being deleted via CASCADE.

C. Exception Handling & Logging Configuration

1. What was implemented:

Custom REST Framework exception handler custom_exception_handler to format errors, catch ProtectedError (preventing deletion of linked records), log full stack traces internally with request_id, and hide internal error details from the client.

2. Files Modified:

- clinic_backend/audit/exceptions.py (Lines 1-43)
- clinic_backend/clinic_backend/settings.py (Line 195)

3. Exact Code Location & Snippet:

Function: custom_exception_handler in audit/exceptions.py | Lines: 10–40

```
def custom_exception_handler(exc, context):
    if isinstance(exc, ProtectedError):
        return Response({"error": "This item cannot be deleted because it is referenced by other important records."},
            status=status.HTTP_400_BAD_REQUEST)

    response = exception_handler(exc, context)

    if response is None:
        request = context.get('request')
        request_id = getattr(request, 'request_id', 'unknown')
        logger.error(f"Unhandled Exception [Request ID: {request_id}]: {str(exc)}\n{traceback.format_exc()}")
        return Response({"error": "Unable to process request", "request_id": request_id}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)

    return response
```

4. Flow Explanation:

Unhandled Exception occurs in view → Caught by custom_exception_handler → Stack trace logged internally with request_id → Clean JSON returned to user containing request_id and generic error message.

5. Why this code was added (Real-World HMS Justification):

Prevents sensitive backend implementation details or DB structure from leaking in HTTP responses while giving developers an exact request_id to look up the full stack trace in server logs.

6. Before vs After:

Before: Unhandled 500 exceptions returned HTML debug pages or raw exception strings.

After: Standardized JSON error response with request_id correlation.

7. Examiner Viva Questions & Answers:

Q: Why catch ProtectedError specifically in the exception handler?

A: Django raises ProtectedError when trying to delete a model referenced by a on_delete=models.PROTECTED foreign key (e.g. deleting a Doctor who has patient prescriptions). Catching it allows returning a clean 400 error.

TOPIC 6: TESTING

A. Unit Testing Setup & Database Isolation

1. What was implemented:

Configured Django test runner detection in settings.py to automatically switch to an in-memory/file-based isolated SQLite database (test_db.sqlite3) when running python manage.py test.

2. Files Modified:

- clinic_backend/clinic_backend/settings.py (Lines 126-133)

3. Exact Code Location & Snippet:

File: clinic_backend/settings.py | Lines: 126–133

```
import sys
if "test" in sys.argv:
    DATABASES = {
        "default": {
            "ENGINE": "django.db.backends.sqlite3",
            "NAME": BASE_DIR / "test_db.sqlite3",
        }
    }
```

4. Flow Explanation:

Command python manage.py test → sys.argv contains 'test' → Django overrides DATABASES['default'] to SQLite → Runs test suite without wiping or connecting to production PostgreSQL.

5. Why this code was added (Real-World HMS Justification):

Unit tests must run fast and must never mutate or truncate development or production PostgreSQL databases.

6. Before vs After:

Before: Running tests risked executing against the active PostgreSQL database specified in .env.

After: Automatic detection ensures complete database isolation during test runs.

7. Examiner Viva Questions & Answers:

Q: How do you run tests for specific apps in Django?

A: python manage.py test accounts appointments audit

B. Authentication, Permission, Appointment & Audit Test Coverage

1. What was implemented:

Test suites covering user registration, JWT obtain/refresh/logout, RBAC authorization, slot constraint enforcement, and audit middleware logging.

2. Files Modified:

- clinic_backend/audit/tests.py
- clinic_backend/beds/tests.py

3. Exact Code Location & Snippet:

Execution Command: python manage.py test

```
# Run test suite across backend apps:
python manage.py test

# Test coverage breakdown:
1. Authentication Tests: User registration (201), JWT issue, HttpOnly cookie set, Cookie refresh rotation, Logout blacklisting.
2. Permission Tests: Patient access to doctor-only endpoints (403), Doctor creation by admin (201).
3. Appointment Tests: Slot booking (201), Concurrent duplicate booking (400 validation error caught from IntegrityError).
4. Audit Tests: X-Request-ID propagation & AuditEvent DB creation.
```

4. Flow Explanation:

Test Runner execution → Sets up isolated test DB → Executes API test cases using DRF APITestCase client → Asserts HTTP status codes & database record changes → Tears down test DB.

5. Why this code was added (Real-World HMS Justification):

Automated testing ensures that future modifications or additions to the codebase do not break existing security controls, authorization boundaries, or appointment integrity rules.

6. Before vs After:

Before: Manual API testing via Postman without automated regression checks.

After: Executable test commands that programmatically verify all security, authentication, authorization, and appointment constraints.

7. Examiner Viva Questions & Answers:

Q: What test client is used for DRF API testing?

A: rest_framework.test.APIClient or APITestCase, which supports helper methods like client.force_authenticate(user=user) and cookie inspection.

SUMMARY VIVA CHECKLIST TABLE

Topic	Key File Path	Class / Function / Setting	Key Highlight for Examiner
Security Foundation	clinic_backend/settings.py	decouple.config AUTH_PASSWORD_VALIDATORS	Key separation, .env validation, security headers, rate limiting
Authentication	accounts/cookie_utils.py accounts/views.py	set_auth_cookie CookieTokenRefreshView LogoutView	HttpOnly cookies, 15-min JWT, token rotation, token blacklisting
Authorization	accounts/permissions.py appointments/views.py	IsAdmin IsAppointmentParticipant get_queryset	RBAC, IDOR protection, role-based DB queryset filtering

Appointment Integrity	appointments/models.py appointments/views.py	UniqueConstraint perform_create transaction.atomic	Conditional slot constraint, row locking, IntegrityError handling
Audit Logging	audit/middleware.py audit/exceptions.py	AuditMiddleware custom_exception_handler	Request UUID, X-Request-ID, DB audit model, exception sanitization
Testing	clinic_backend/settings.py	if 'test' in sys.argv:	SQLite DB test isolation & DRF unit test execution